CURSO:
ESTRUCTURA DE DATOS CON POO

Docente: M(o). Ing. Loncán Salazar, Pierre Paul

SESIÓN Nº 09

AGENDA:

- Árbol (Parte I)

ÁRBOLES

- Árbol es el nombre que se le da a un grupo versátil de estructuras de datos.
- Se pueden utilizar para implementar un número de interfaces abstractas, incluida la interfaz List, pero las aplicaciones en las que resultan más útiles emplean estructuras de ramas de árboles para representar alguna propiedad de los elementos de los datos o para optimizar ciertos métodos.

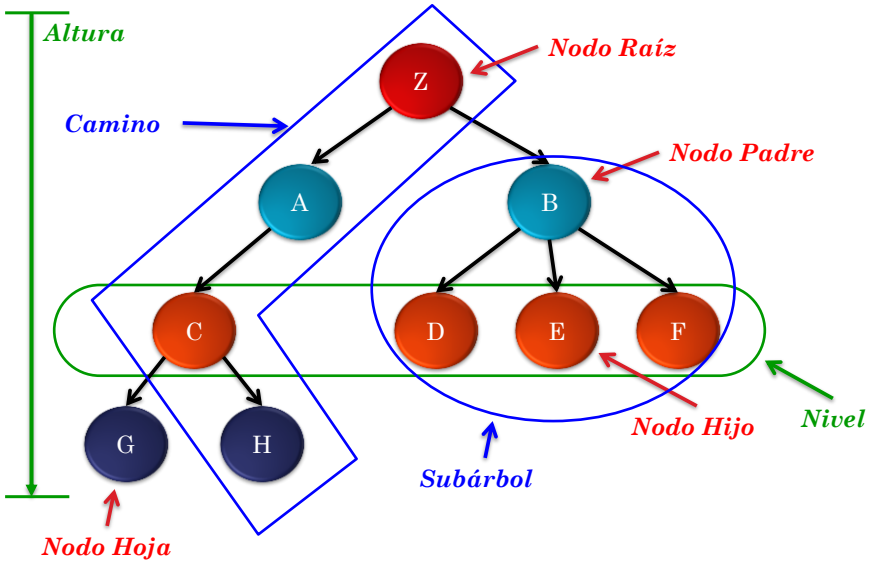
M(o). Ing. Loncán Salazar, Pierre Paul

TERMINOLOGÍA DE ÁRBOL

- **Raíz:** El padre de todos los nodos.
- **Padre:** Nodo anterior a un nodo.
- **Hijo:** Nodo al que otro nodo está apuntando (su padre)
- **Hoja:** Nodo sin hijos, en estos nodos es en los que vamos a realizar la inserción.
- **Subárbol:** Subconjunto del árbol.
- **Camino:** Recorrido de un nodo a otro.
- **Niveles:** Nodos que están al mismo nivel si están a la misma profundidad en el árbol.
- **Altura:** Número total de niveles.

M(o). Ing. Loncán Salazar, Pierre Paul

TERMINOLOGÍA DE ÁRBOL



M(o). Ing. Loncán Salazar, Pierre Paul

ÁRBOLES, NODOS Y RAÍCES

- Un árbol consta de nodos conectados.
- Cada árbol (salvo un árbol vacío) cuenta con un nodo distinguido llamado raíz.
- No puede haber rutas circulares en las conexiones de un árbol, de tal forma que sólo puede existir una ruta o camino único desde cada nodo hasta la raíz.

M(o). Ing. Loncán Salazar, Pierre Paul

Nodos hijos y padres

- Todos los nodos conectados a un nodo concreto son hijos o bien el padre de dicho nodo.
- Si el nodo conectado se encuentra en la única ruta a la raíz, dicho nodo recibe el nombre de padre. Todos los nodos, salvo la raíz, tienen un único padre.
- El resto de nodos conectados a un nodo concreto son los hijos del nodo

M(o). Ing. Loncán Salazar, Pierre Paul

7

Antecesores, descendientes y subárboles

- Los nodos que se encuentran en la ruta que va desde un nodo a la raíz reciben el nombre de antecesores del nodo e incluyen a su padre, al padre de su padre, etc., hasta llegar a la raíz.
- El conjunto de nodos que incluyen a los hijos del nodo, a los hijos del hijo, etc., reciben el nombre de descendientes del nodo.
- Un nodo y sus descendientes forman un subárbol enraizado a dicho nodo.
- Un nodo sin hijos recibe el nombre de hoja

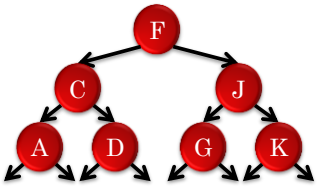
M(o). Ing. Loncán Salazar, Pierre Paul

8

ÁRBOLES BINARIOS

ÁRBOL BINARIO

- Tipo de árbol habitual y de gran utilidad que permite que un nodo tenga, al menos, dos hijos.
- Definición formal del árbol binario:
  - ***Un árbol binario es un árbol vacío, o bien un nodo raíz con subárboles formados por árboles binarios a la derecha y a la izquierda***.



CLASE ARBOL (CÓDIGO)

```
public class Arbol
{
    private Nodo_Agenda Raiz;

    public Arbol()
    {
        Raiz = null;
    }
}
```

Clase Persona (Código)

```
public class Persona
{
    private String Apellidos;
    private String Nombres;
    private String Telefono;

    public Persona(Object[] Registo)
    {
        this.Apellidos = Registo[0].toString();
        this.Nombres = Registo[1].toString();
        this.Telefono = Registo[2].toString();
    }

    public Object[] getRegistro()
    {
        Object[] Registro = {Apellidos, Nombres, Telefono};
        return Registro;
    }

    public String getNomCompleto() {return Apellidos + " " + Nombres;};

    public String getApellidos() {return Apellidos;}
    public void setApellidos(String Apellidos) {this.Apellidos = Apellidos;}

    public String getNombres() {return Nombres;}
    public void setNombres(String Nombres) {this.Nombres = Nombres;}

    public String getTelefono() {return Telefono;}
    public void setTelefono(String Telefono) {this.Telefono = Telefono;}
}
```


Clase Nodo (Código)

```
public class Nodo_Agenda
{
    private Persona Elemento;
    private Nodo_Agenda Izq, Der;

    public Nodo_Agenda(Persona Elemento)
    {
        this.Elemento = Elemento;
        Izq = Der = null;
    }

    public Nodo_Agenda getDer() {return Der;}
    public void setDer(Nodo_Agenda Der) {this.Der = Der;}

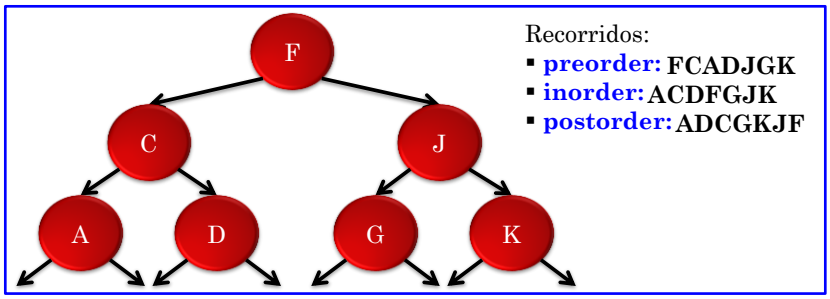
    public Persona getElemento() {return Elemento;}
    public void setElemento(Persona Elemento) {this.Elemento = Elemento;}

    public Nodo_Agenda getIzq() {return Izq;}
    public void setIzq(Nodo_Agenda Izq) {this.Izq = Izq;}

}
```

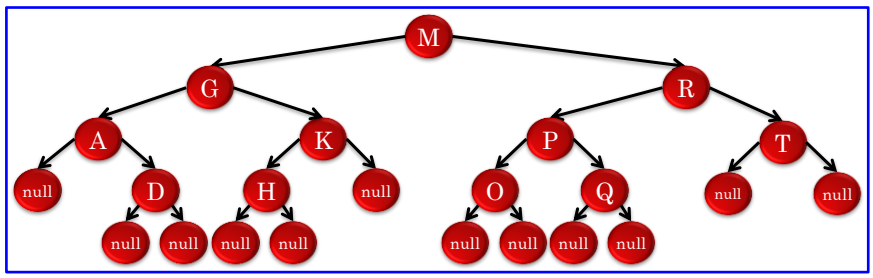
Recorrido del árbol

- Los tres métodos más comunes para recorrer un árbol se pueden describir, de forma recursiva, del siguiente modo:
  - **preorder:** raíz, subárbol izquierdo, subárbol derecho.
  - **inorder:** subárbol izquierdo, raíz, subárbol derecho.
  - **postorder:** subárbol izquierdo, subárbol derecho, raíz.



M(o). Ing. Loncán Salazar, Pierre Paul

Códigos de ejemplo de recorrido (Modalidad Recursiva)




```
void preorden (Nodo Raiz)
{
  if (Raiz!=null)
  {
     Imprimir(Raiz.info);
     preorden(Raiz.izq);
     preorden(Raiz.der);
  }
}
```

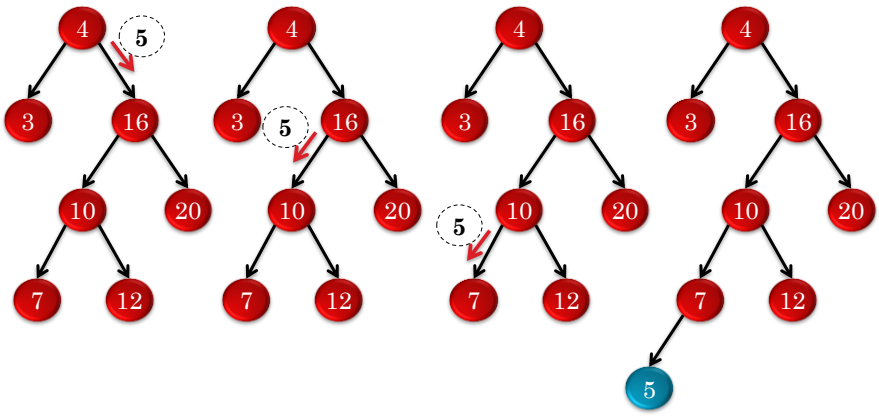
```
void inorden (Nodo Raiz)
{
  if (Raiz!=null)
  {
     inorden(Raiz.izq);
     Imprimir(Raiz.info);
     inorden(Raiz.der);
  }
}
```

```
void posorden (Nodo Raiz)
{
  if (Raiz!=null)
  {
     posorden(Raiz.izq);
     posorden(Raiz.der);
     Imprimir(Raiz.info);
  }
}
```

M(o). Ing. Loncán Salazar, Pierre Paul

INSERCIÓN DE NODOS

Valor a Insertar → 5



M(o). Ing. Loncán Salazar, Pierre Paul

15

INSERCIÓN DE NODOS (CÓDIGO)

```
public Nodo_Agenda Agregar(Nodo_Agenda Nodo, Persona Elemento)
    {
        // Evalua si el nodo donde se pretende agregar el elemento es Nulo
        if (Nodo == null)
        {
            Nodo_Agenda Nuevo = new Nodo_Agenda(Elemento); // Crea el nodo
            return Nuevo; // retorna el nodo creado
        }
        else
        {
            // Evalua si el nombre completo del Nodo
            // es menor al Nombre Completo del Elemento
            if (Nodo.getElemento().getNomCompleto().
                compareTo(Elemento.getNomCompleto())<0)
            {
                // Agrega el Elemento en el enlace derecho del Nodo
                // y luego con el valor retornado por el método Agregar
                // se actualiza el enlace derecho del Nodo
                Nodo.setDer(Agregar(Nodo.getDer(), Elemento));
            }
            else
            {
                // Agrega el Elemento en el enlace izquierdo del Nodo
                // y luego con el valor retornado por el método Agregar
                // se actualiza el enlace izquierdo del Nodo
                Nodo.setIzq(Agregar(Nodo.getIzq(), Elemento));
            }
        }
        return Nodo; // Luego de modificar los enlaces el Nodo es retornado

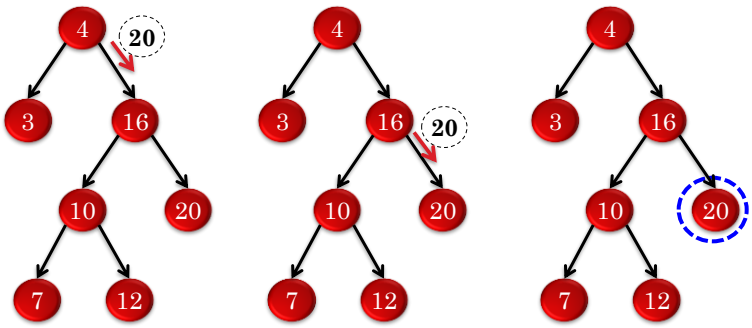
    }
```

M(o). Ing. Loncán Salazar, Pierre Paul

16

Búsqueda de un Valor

Valor Buscado → 20



En caso no se encontrara el valor se retornaría null

M(o). Ing. Loncán Salazar, Pierre Paul

17

Búsqueda de valores (Código)

```
public Nodo_Agenda BuscarApeNom(String Dato)
    {
        // Inicia la Búsqueda por el nodo raiz
        Nodo_Agenda Auxiliar = Raiz;

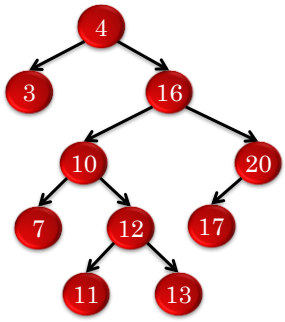
        // Mientras Auxiliar no sea nulo continúa la búsqueda
        while (Auxiliar != null)
        {
            // Compara un valor del Auxiliar con el Dato Buscado
            if (Auxiliar.getElemento().getNomCompleto().equals(Dato))
            {
                // Retorna el nodo Auxiliar si se encuentra coincidencia
                return Auxiliar;
            }
            else
            {
                // Evalúa para determinar si debe continuar la búsqueda
                // por el nodo de la derecha o por el nodo de la izquierda
                if (Dato.compareTo(Auxiliar.getElemento().getNomCompleto())>0)
                    Auxiliar = Auxiliar.getDer();
                else
                    Auxiliar = Auxiliar.getIzq();
            }
        }
        // Retorna nulo cuando recorre todo el árbol sin encontrar el valor
        return null;
    }
```

M(o). Ing. Loncán Salazar, Pierre Paul

18

Eliminar un Valor

- Pueden existir 3 casos de eliminación:
  - Caso 1: El nodo eliminado no tiene hijos (Ejm: Nodo 7)
  - Caso 2: El nodo eliminado tiene un hijo (Ejm: Nodo 20)
  - Caso 3: El nodo eliminado tiene dos hijos (Ejm: Nodo 16)

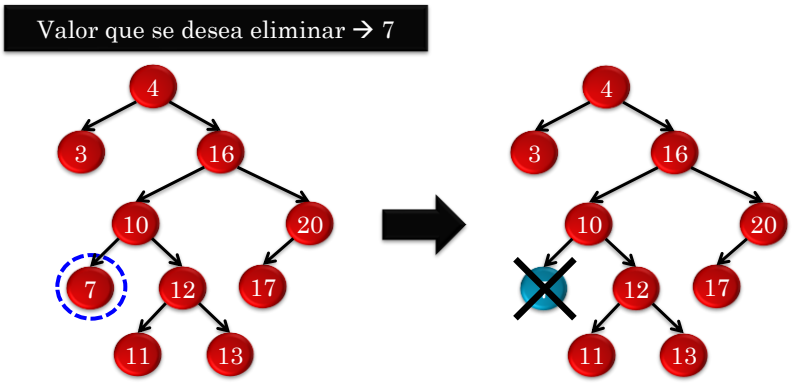


M(o). Ing. Loncán Salazar, Pierre Paul

19

Caso 1: Sin Hijos

- Pasos:
  - Localizar el nodo con el valor deseado
  - Al no tener hijos, sólo se requiere asignar el valor **null** al enlace Izquierda o Derecha del Nodo Padre

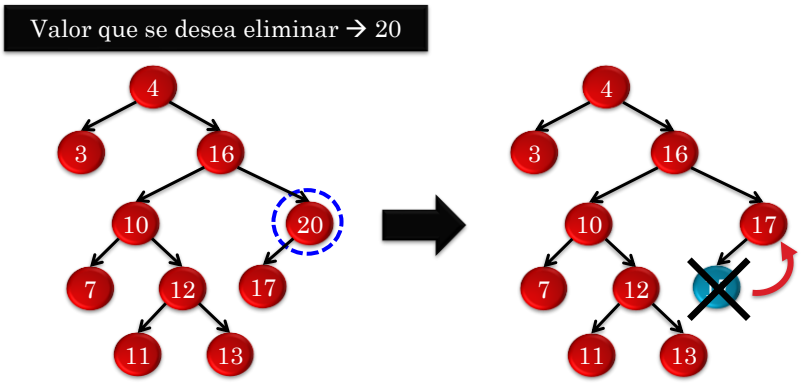


M(o). Ing. Loncán Salazar, Pierre Paul

20

Caso 2: Con 1 Hijo

- Pasos:
  - Localizar el nodo con el valor deseado
  - Al sólo tener un solo hijo, se copiará el contenido del nodo hijo en el nodo padre.
  - Luego se eliminará el nodo asignando el valor null al enlace Izquierda o Derecha del Nodo Padre

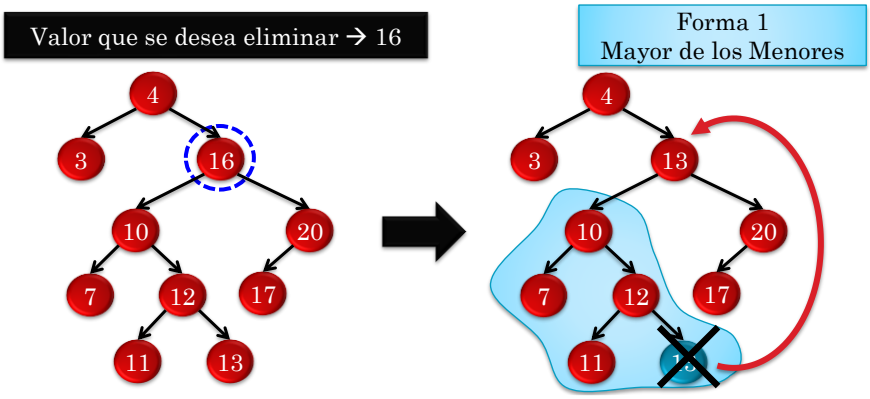


M(o). Ing. Loncán Salazar, Pierre Paul

21

Caso 3: Con 2 Hijos

- Pasos:
  - Localizar el nodo con el valor deseado
  - Se puede proceder de 2 formas:
    - **Forma 1 (Mayor de los Menores):** Buscar y sustituir el nodo encontrado con el MAYOR valor del sub-arbol izquierdo (Valores Menores)
    - **Forma 2 (Menor de los Mayores):** Buscar y sustituir el nodo padre con el MENOR valor del sub-arbol derecho (Valores Mayores)

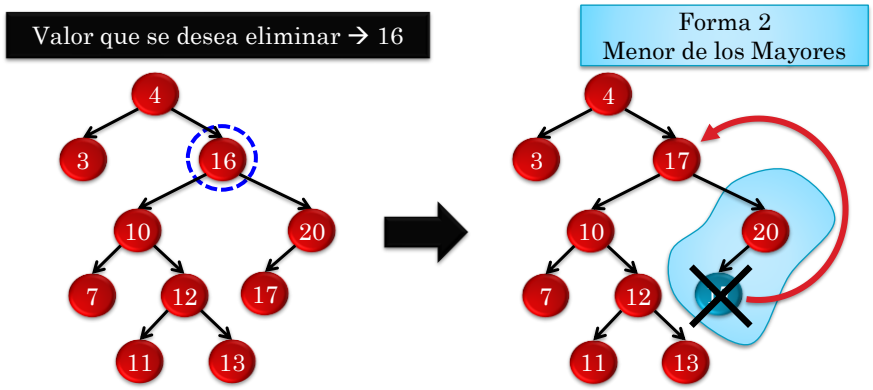


M(o). Ing. Loncán Salazar, Pierre Paul

22

Caso 3: Con 2 Hijos

- Pasos:
  - Localizar el nodo con el valor deseado
  - Se puede proceder de 2 formas:
    - **Forma 1 (Mayor de los Menores):** Buscar y sustituir el nodo encontrado con el MAYOR valor del sub-arbol izquierdo (Valores Menores)
    - **Forma 2 (Menor de los Mayores):** Buscar y sustituir el nodo padre con el MENOR valor del sub-arbol derecho (Valores Mayores)



M(o). Ing. Loncán Salazar, Pierre Paul

Eliminar valores (Código)

```
public Nodo_Agenda BuscarMayorIzquierda(Nodo_Agenda Auxiliar)
    {
        // Auxiliar es el Nodo Izquierda del Nodo a eliminar
        if (Auxiliar != null)
        {
            // Mientras no sea null continúa por la derecha
            while (Auxiliar.getDer() != null)
            {
                Auxiliar = Auxiliar.getDer();
            }
        }
        // Retorna el Nodo encontrado
        // (Es el nodo Mayor de la Subrama derecha)
        return Auxiliar;
    }
public Nodo_Agenda EliminarMayorIzquierda(Nodo_Agenda Auxiliar)
    {
        // Auxiliar es el Nodo Izquierda del Nodo a Eliminar
        if (Auxiliar == null)
            return null;
        else if (Auxiliar.getDer() != null)
        {
            // Si la derecha del Auxiliar es diferente de null
            // se llamará recursivamente al método EliminarMayorIzquierda
            // enviando el Nodo derecha
            Auxiliar.setDer(EliminarMayorIzquierda(Auxiliar.getDer()));
            // Al retornar de la recusividad se actualiza el enlace derecho
            // del Auxiliar y se retorna en Nodo Auxiliar
            return Auxiliar;
        }
        // Cuando el Enlace derecho sea null se retornará en enlace
        // Izquierdo
        return Auxiliar.getIzq();
    }
```

M(o). Ing. Loncán Salazar, Pierre Paul

ELIMINAR VALORES (CÓDIGO)

```
public Nodo_Agenda Eliminar(Nodo_Agenda Auxiliar, String Dato)
{
    if (Auxiliar == null)
        return null;

    if (Dato.compareTo(Auxiliar.getElemento().getNomCompleto()) < 0)
    {
        // Si dato es menor al valor del nodo se llama al método Eliminar
        Auxiliar.setIzq(Eliminar(Auxiliar.getIzq(),Dato));
        // Al retornar de la recursividad se actualiza el Enlace Izquierda
    }
    else if (Dato.compareTo(Auxiliar.getElemento().getNomCompleto()) > 0)
    {
        // Si dato es mayor al valor del nodo se llama al método Eliminar
        Auxiliar.setDer(Eliminar(Auxiliar.getDer(), Dato));
        // Al retornar de la recursividad se actualiza el Enlace Derecha
    }
    else if (Auxiliar.getIzq() != null && Auxiliar.getDer() != null)
    {
        // Se encontró el nodo a eliminar y tiene 2 hijos
        // Se busca el MAYOR de los MENORES y se actualiza Elemento
        Auxiliar.setElemento(BuscarMayorIzquierda(Auxiliar.getIzq()).
                             getElemento());
        // Se elimina el MAYOR de los MENORES
        Auxiliar.setIzq(EliminarMayorIzquierda(Auxiliar.getIzq()));
    }
    else
    {
        // Se encontró el nodo a eliminar, pero tiene un hijo
        // Se actualiza Auxiliar con ese enlace izquierdo o derecho
        Auxiliar = ( Auxiliar.getIzq() != null ) ?
                    Auxiliar.getIzq() : Auxiliar.getDer();
    }
    return Auxiliar;
}
```